HSLU

Hochschule Luzern
Departement Informatik
Student: Björn Pfoster

1 Problem statement

Every morning at 08:00 Swiss time, forecast Switzerland's **total hourly electricity load (MW)** for all 24 hours of the **next calendar day** (horizon 16 - 39 h ahead).

Stakeholders: traders, balance-group managers and energy-aware consumers who plan one day ahead; the target is the same quantity a day-ahead market bid is based on. **Scope:** national load, bidding zone CH, hourly resolution, one forecast run per day.

Baseline and success criterion The baseline is the *seasonal-naive* forecast (load at the same hour one week earlier). On a held-out year (23.09.2025–23.09.2026, 8 782 hours) it reaches **MAPE 7.6 %** (MAE 590 MW); 48 h persistence is worse (MAPE 9.2 %). **Success Definition** A MAPE of $\leq$ **6.5 %** on the same held-out year, i.e. at least 15 % lower than the baseline, and this advantage maintained live (rolling 7-day MAPE below the rolling baseline) once the system runs daily. A preliminary, untuned LightGBM model (calendar, 48 h/168 h lags and archived weather forecasts) already reaches MAPE 6.15 % (MAE 481 MW), so the target is realistic; the project's focus is the automated system around the model.

2 Originality & motivation

Why this problem: Switzerland's *winter electricity gap* is a recurring national topic: in winter the country imports power while demand is highest (mean load Nov–Feb 8.5 GW vs. 7.2 GW in Jun–Aug, 2022–2026). A demand forecast is the first building block for any statement about that gap.

What makes it different: (1) the project runs Oct–Jan, i.e. straight into the heating season, so the system *will* face real, predictable data drift (monthly baseline MAPE ranges from 4.6 % to 10.8 % over the year): the monitoring and retraining parts are tested on a real event, not on a synthetic one; (2) training features are built from *archived weather forecasts* instead of observed weather, so the model is trained on exactly the kind of input it sees in production (no hidden training–serving skew).

3 Data source & features

Load (label)	Energy-Charts API (Fraunhofer ISE), `/public_power?country=ch`, field `Load`. Public REST API, no key, no scraping. Hourly values, updated within a few hours. Fallback: ENTSO-E Transparency Platform (free token).
Weather	Open-Meteo: *Forecast API* for live runs, *Historical Forecast API* (archived forecasts, available from 2022) for backfill. No key, free for non-commercial use. Zurich, Geneva, Basel, Bern.
Volume	Load history available since 2015; used from 01.2022 (start of the forecast archive): 41 447 hourly rows today, no gaps; grows by 24 rows/day ($\approx$ 8 760/year).

3.1 Labels

measured national load of hour t, published by Energy-Charts after the fact. It is a physical measurement, not a rule applied to the features, and it arrives about one day after the forecast, which enables live evaluation. **Class imbalance:** not applicable (regression); public holidays are rare and evaluated separately.

3.2 Named features

(one row per target hour):

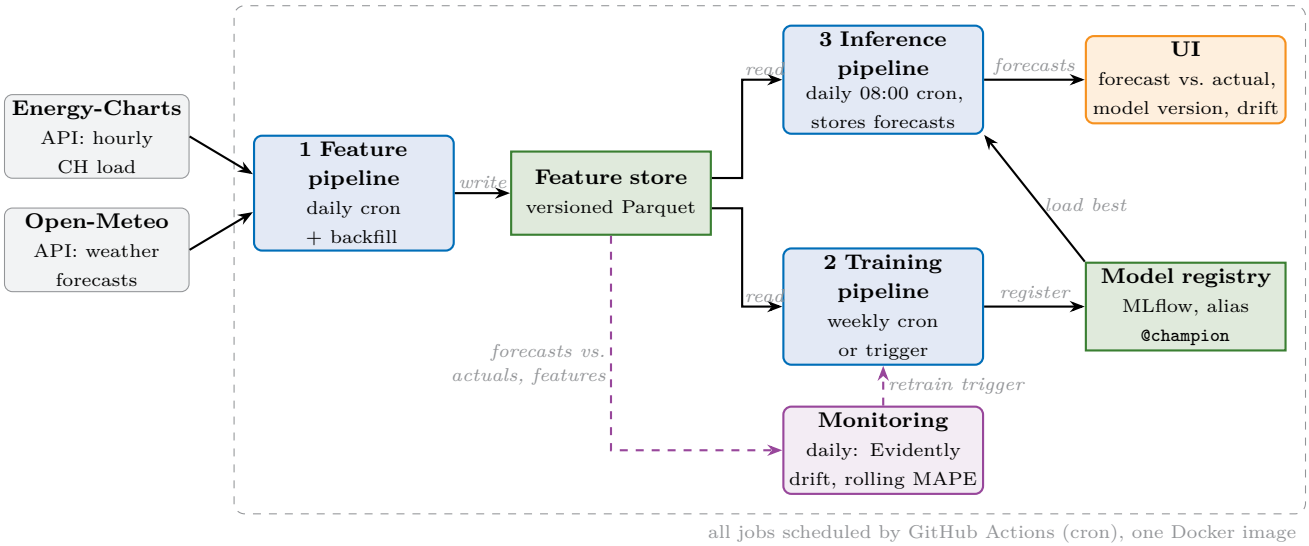
- *Calendar:* `hour_local`, `day_of_week`, `month`, `is_holiday_ch` (Python `holidays` package), known in advance.
- *Autoregressive:* `load_lag_48h`, `load_lag_168h`, `load_mean_7d` (7-day mean ending 48 h before t).

- *Weather forecast for hour t* (mean of the four cities): `temp_2m`, `heating_degree` $= \max(0, 18°\text{C} - $ `temp_2m`$)$, `cloud_cover`, `shortwave_radiation` (rooftop PV lowers the measured load on sunny days).

3.3 Leakage and split

Out-of-time split: train 01.2022–22.09.2025, test 23.09.2025–23.09.2026; tuning only with `TimeSeriesSplit` inside the training period. *Lags:* at 08:00 on day $D-1$ the newest load value is only a few hours old, so no lag shorter than 48 h is used (a lag of 24 h would read values that do not exist yet); a unit test enforces this. *Weather:* using observed weather for training would leak the future; archived forecasts avoid this. *Skew:* feature definitions live in one module used by training and inference.

4 System design



Layer	Choice	Why
Orchestration	GitHub Actions (cron)	Free, versioned with the code, no scheduler server to run.
Feature store	Parquet in a Hugging Face Dataset repo	Every write is a commit, so every feature version is kept; free; accepted "poor man's" store.
Tracking & registry	MLflow, hosted on DagsHub	Tool used in the course; aliases give a clear champion/challenger promotion; no own server.
Model	LightGBM regressor	Strong on tabular data, trains in seconds inside a CI runner.
Serving & UI	Streamlit in Docker on HF Spaces	Free cloud deployment; same Docker image as the pipelines.
Monitoring	Evidently + rolling MAPE	Feature drift before labels arrive, real error once they do.
Quality	`uv.lock`, Dockerfile, `pytest` in CI	Pinned, reproducible, tested (no-leakage and no-gap tests).

Retraining: weekly cron, plus an extra run when the 7-day MAPE exceeds 8 % or Evidently flags feature drift; a new model becomes `@champion` only if it beats the current one on the most recent 8 weeks. **Optional stretch goals** (only after the core FTI system runs): Hopsworks instead of Parquet, a FastAPI prediction endpoint, a Grafana dashboard, probabilistic (quantile) forecasts. **The repository is public** at https://github.com/D4rkMindz/hslu-winterload.